SKILL STACK

Everything you're standing on, from the bedrock up.

Nine layers, read top to bottom. **Layer 0 is the foundation everything above it sits on** — you don't need to master it before touching Layer 3, but every layer gets easier once the one below it isn't a total mystery. Nobody holds all of this at expert depth; the goal is no layer being a total blind spot. Check things off as you get comfortable — saved locally in this browser.

0 / 51 internalized

L0 · Bedrock — how a computer actually works
0 / 7

L1 · Programming foundations
0 / 5

L2 · Engineering practice
0 / 6

L3 · Building real applications
0 / 7

L4 · Production engineering
0 / 5

L5 · System design & architecture
0 / 6

L6 · AI/ML specialization
0 / 5

L7 · Working with other humans
0 / 5

L8 · The long game
0 / 5

L0 Bedrock — how a computer actually works

Everything above this assumes you have at least a rough model of what's happening underneath. You don't need to be a hardware engineer — just not mystified.

- [ ] CORE **Binary & number systems**
  All data — text, images, this checkbox — ultimately reduces to 1s and 0s. Knowing why explains a lot of weird edge cases later (like float rounding).

- [ ] CORE **CPU & memory basics**

Registers, RAM, cache — why 'memory access is slow' is the reason so much of performance engineering exists.

- [ ] CORE **Operating system fundamentals**
  Processes vs threads, scheduling, virtual memory — what's actually happening when you run a program.

- [ ] CORE **File systems**
  What 'saving a file' really does on disk, and why it can still fail halfway through.

- [ ] CORE **Networking basics**
  TCP/IP, ports, packets — the plumbing every API call travels through.

- [ ] CORE **The command line**
  Navigating, piping, scripting without a GUI — the default interface for real engineering work.

- [ ] CORE **How the internet works**
  DNS, routing, the client-server model — what actually happens between typing a URL and seeing a page.

L1 Programming foundations

The stuff that transfers between languages — learn it once, deeply, in whichever language you use most (Python, in your case).

- [ ] CORE **One language, deeply**
  Better to fully know one language's quirks than to shallowly know five. Depth here pays off everywhere else.

- [ ] CORE **Data structures**
  Arrays, linked lists, stacks, queues, hash maps, trees, graphs — the vocabulary for describing how data is organized.

- [ ] CORE **Algorithms & complexity**

Sorting, searching, recursion, and reasoning about Big-O — why some solutions get slow and others don't.

- [ ] CORE **Memory management**
  Stack vs heap, garbage collection, references vs values — why mutating a list inside a function sometimes surprises you.

- [ ] CORE **Debugging**
  Reading a stack trace properly, using a real debugger — not just sprinkling print statements and hoping.

L2 Engineering practice

The habits that separate 'code that runs' from code a team can actually maintain.

- [ ] PRACTICE **Version control (Git)**
  Branching, merging, rebasing, resolving conflicts — the shared memory of every team codebase.

- [ ] PRACTICE **OOP & functional thinking**
  When each paradigm actually helps you, not dogma applied for its own sake.

- [ ] PRACTICE **Design patterns**
  Factory, strategy, observer, and the rest — plus knowing when NOT to reach for one.

- [ ] PRACTICE **Clean code / SOLID principles**
  Writing for the next person who reads it (often you, in six months) — not just for the compiler.

- [ ] PRACTICE **Testing**
  Unit, integration, end-to-end — what's worth testing, and what's just busywork.

- [ ] PRACTICE **Code review**
  Giving feedback that improves the code, and receiving it without taking it personally.

L3 Building real applications

Where most of what we've already covered on Leora's backend lives — APIs, auth, data, and how the pieces of an app talk to each other.

- [ ] APP **APIs & protocols**
  REST, GraphQL, gRPC — different shapes for the same core idea: letting two systems talk.

- [ ] APP **Auth & security fundamentals**
  JWT, CORS, password hashing, secrets management — everything from our earlier conversations, in practice.

- [ ] APP **Databases**
  SQL vs NoSQL, indexing, transactions, normalization, ORMs — where your data actually lives and how fast you can get it back.

- [ ] APP **Caching**
  What's worth caching, how it gets invalidated, and where it sits in the request path.

- [ ] APP **Async work & message queues**
  Background jobs, pub/sub — why not everything should block a request while it runs.

- [ ] APP **Frontend basics**
  Enough HTML/CSS/JS to understand how the other half of the app thinks, even as a backend-focused engineer.

- [ ] APP **Build tools & package management**
  Dependency resolution, lockfiles — why 'it works on my machine' happens and how to stop it.

L4 Production engineering

Getting code from your machine into something real users hit reliably. There's already a full checklist for this layer: Production Field Guide.

- [ ] OPS **CI/CD**

Tests and deploys that run automatically on every change, not by hand.

- [ ] OPS **Containers**
  Docker: packaging 'works on my machine' into something portable and reproducible.

- [ ] OPS **Cloud fundamentals**
  Compute, storage, and networking basics on AWS/GCP/Azure — the building blocks everything else rents.

- [ ] OPS **Observability**
  Logging, metrics, tracing — knowing what's actually happening in production, not guessing.

- [ ] OPS **Infrastructure as code**
  Environments defined in version-controlled files (Terraform etc.), not manual console clicks.

L5 System design & architecture

The layer where 'does it work' stops being enough and 'does it survive scale, load, and failure' takes over.

- [ ] DESIGN **Scaling patterns**
  Load balancing, replication, sharding — horizontal vs vertical, and when each makes sense.

- [ ] DESIGN **CAP theorem & consistency trade-offs**
  Why a distributed system can't have perfect consistency, availability, and partition tolerance all at once.

- [ ] DESIGN **Microservices vs monoliths**
  Real trade-offs — not 'microservices are always better,' which is usually wrong for a small team.

- [ ] DESIGN **Event-driven architecture**
  Decoupling systems through events instead of direct calls, so one slow part doesn't stall everything.

- [ ] DESIGN **Designing for failure**
  Timeouts, retries, circuit breakers, graceful degradation — assuming things WILL break, and planning for it.

- [ ] DESIGN **Trade-off thinking**
  The real senior-engineer skill: every architectural choice costs something — the job is choosing which cost you can afford.

L6 AI/ML specialization

Directly relevant to Leora — the layer on top of standard backend work once an LLM is part of the product.

- [ ] AI **Prompt engineering**
  Structuring inputs to reliably get the output shape you actually want from a model.

- [ ] AI **Embeddings & vector search**
  How 'semantic search' works under the hood — turning meaning into comparable numbers.

- [ ] AI **Retrieval-Augmented Generation**
  Grounding an LLM's answers in your own data instead of only what it memorized during training.

- [ ] AI **Evaluating AI output**
  Since outputs are non-deterministic, you need a deliberate way to catch quality regressions.

- [ ] AI **Cost & token management**
  Treating LLM calls as a metered, fallible external resource — not a free, always-on function.

L7 Working with other humans

Code is rarely the bottleneck past a certain point — communication and judgment are.

- [ ] TEAM **Writing clearly**
  PR descriptions, docs, tickets precise enough that they don't need a follow-up meeting to decode.

- [ ] TEAM **Estimation & scoping**
  Turning an ambiguous ask into something you can actually commit to and be right about.

- [ ] TEAM **Agile/Scrum basics**
  Sprints, standups, backlogs — the shared vocabulary of how most teams organize work.

- [ ] TEAM **Reading unfamiliar code**
  Navigating a codebase you didn't write, without needing the original author in the room.

- [ ] TEAM **Mentoring**
  The skill that raises your ceiling once you're past mid-level — your impact stops being just your own output.

L8 The long game

Not a skill you finish — a set of habits that compound. This layer never gets checked off completely, and that's the point.

- [ ] MINDSET **Build real projects**
  One finished side project teaches more than ten tutorials — Leora itself counts.

- [ ] MINDSET **Read other people's code**
  Study a mature open-source codebase, not just its docs — see how real trade-offs got made.

- [ ] MINDSET **Ship and get feedback**
  Real usage reveals problems no to-do-list side project ever will.

- [ ] MINDSET **Depth in one area, breadth everywhere else**
  Be the go-to person on something specific; stay conversant in the rest.

MINDSET **Know what you don't know**

The senior/junior gap is mostly about knowing which questions to ask, not having every answer memorized.

How to actually use this: don't try to fill every box before building anything. Get L0–L3 solid, then build real things and let L4 onward fill in as you hit them — you already started that with Leora's auth/CORS work and the B2B API example.

Nobody is fully checked on every layer, including senior engineers. The list stops mattering once you stop needing it as a map.